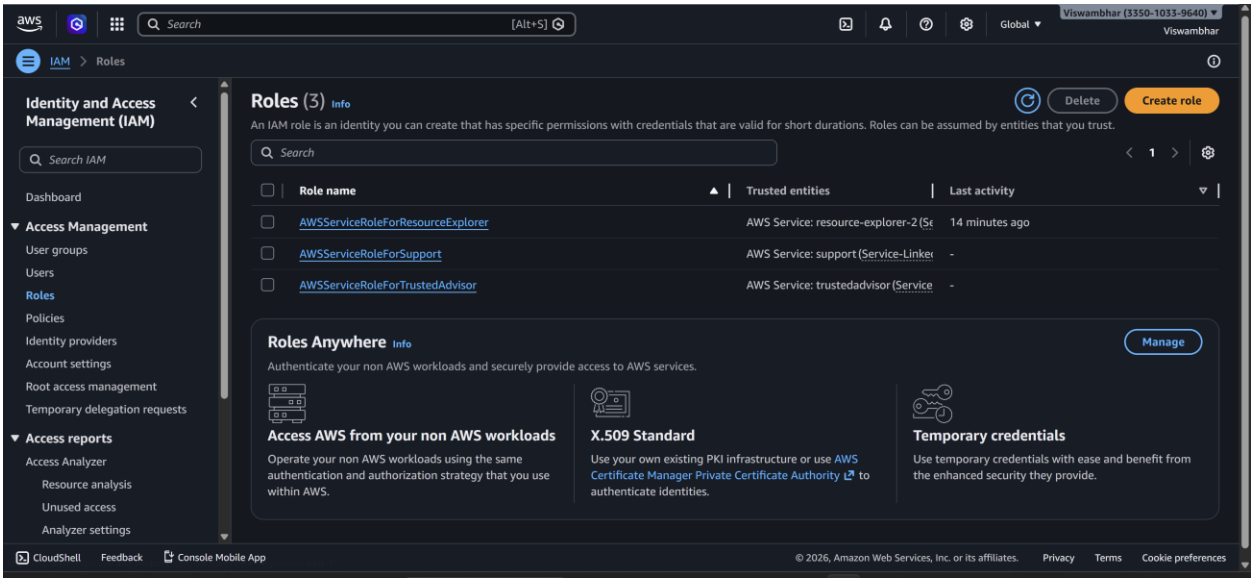
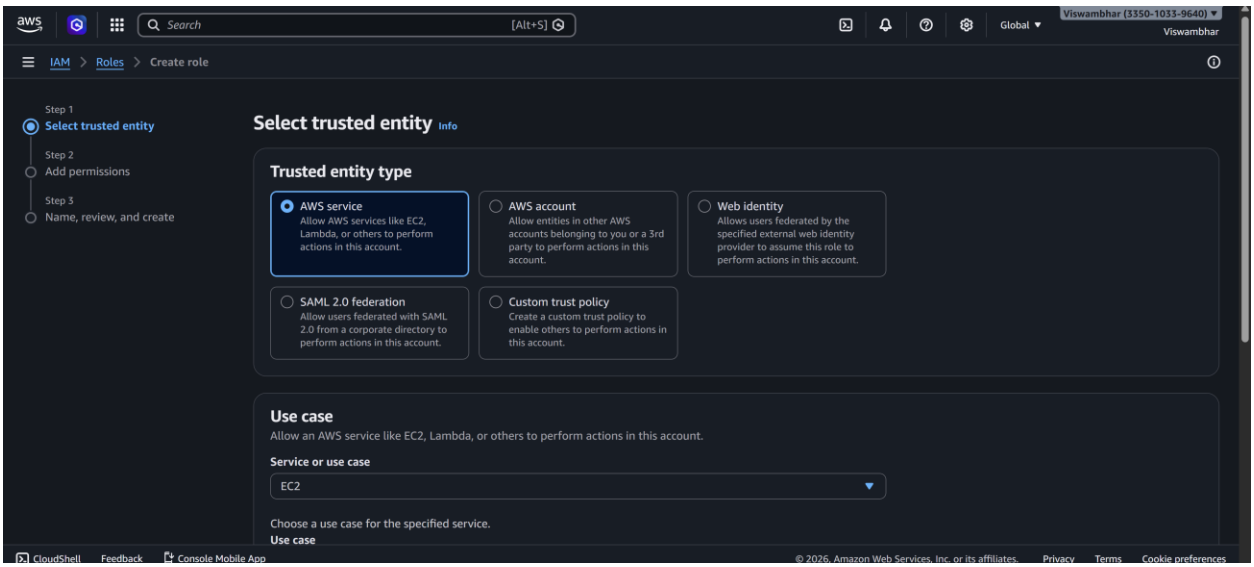
Week-13 Procedure: Attach an IAM Role to an EC2 Instance (S3 access without keys)

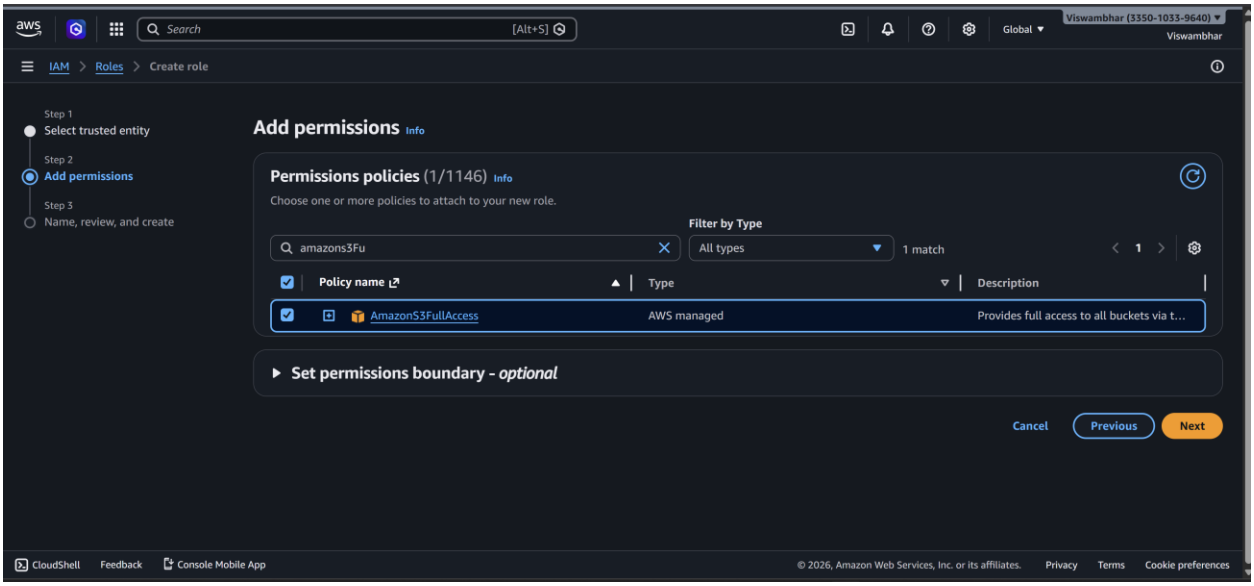
Step 1 — Create Role in Amazon Web Services IAM

1. Open IAM Console → Roles → Create role
2. 

3. Trusted entity: Select AWS Service
4. Use case: Choose EC2
5. Click Next
6. 

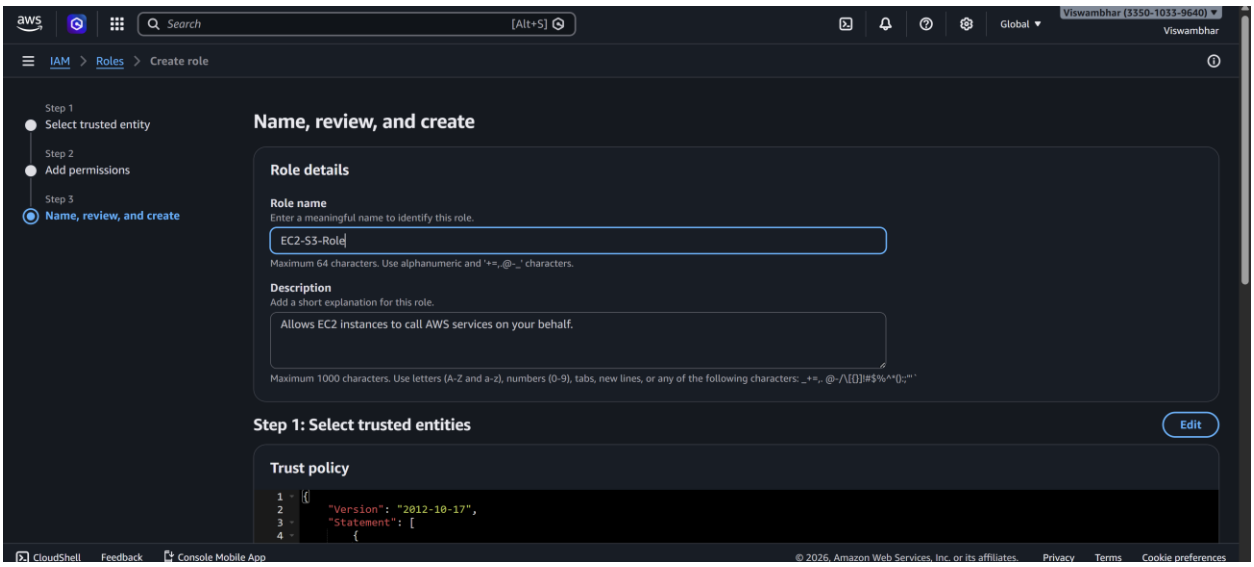
Step 2 — Add Permissions

1. Search policy: AmazonS3FullAccess
2. Select it → Next



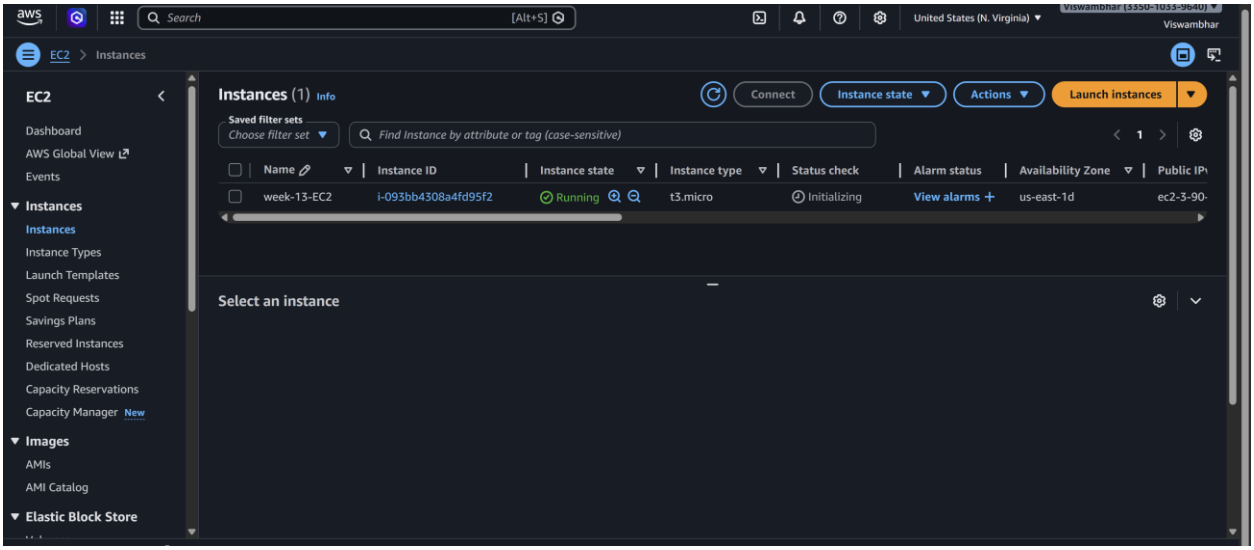
Step 3 — Name the Role

- Name: EC2-S3-Role
- Click Create role
-



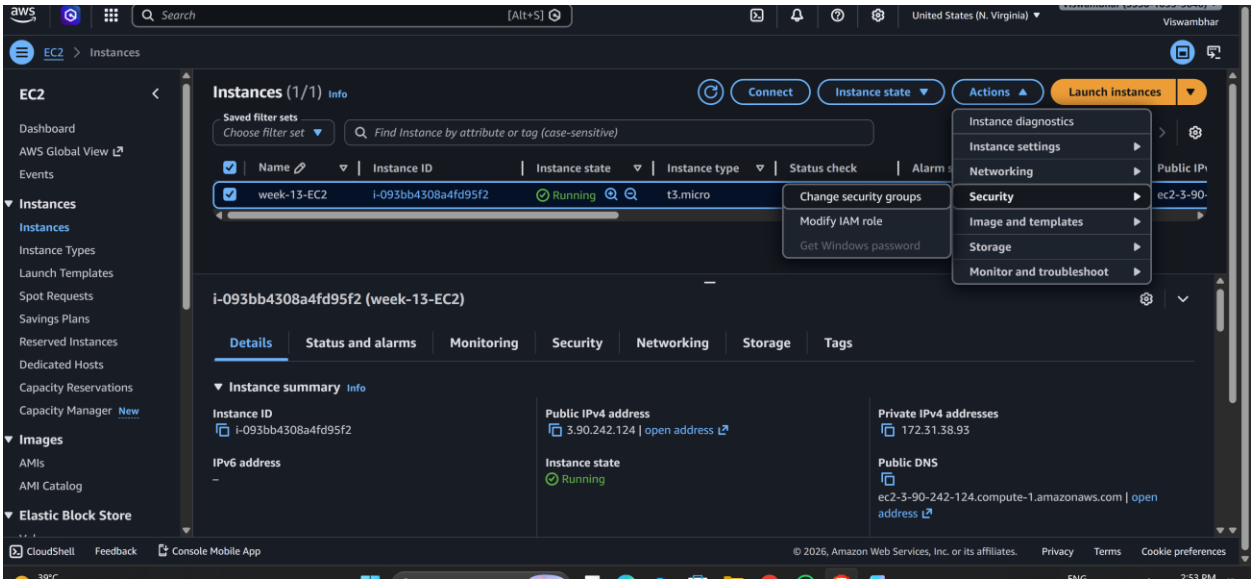
Step 4 — Attach Role to EC2 Instance

1. Go to EC2 Dashboard

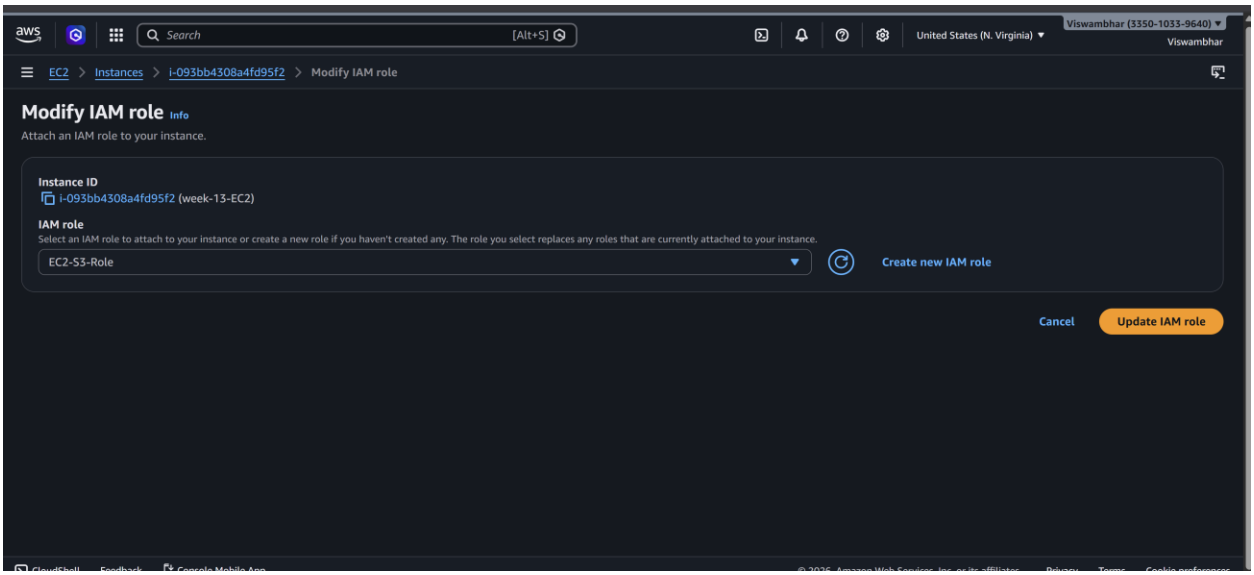
2. 

3. Select running instance

4. Actions → Security → Modify IAM role

5. 

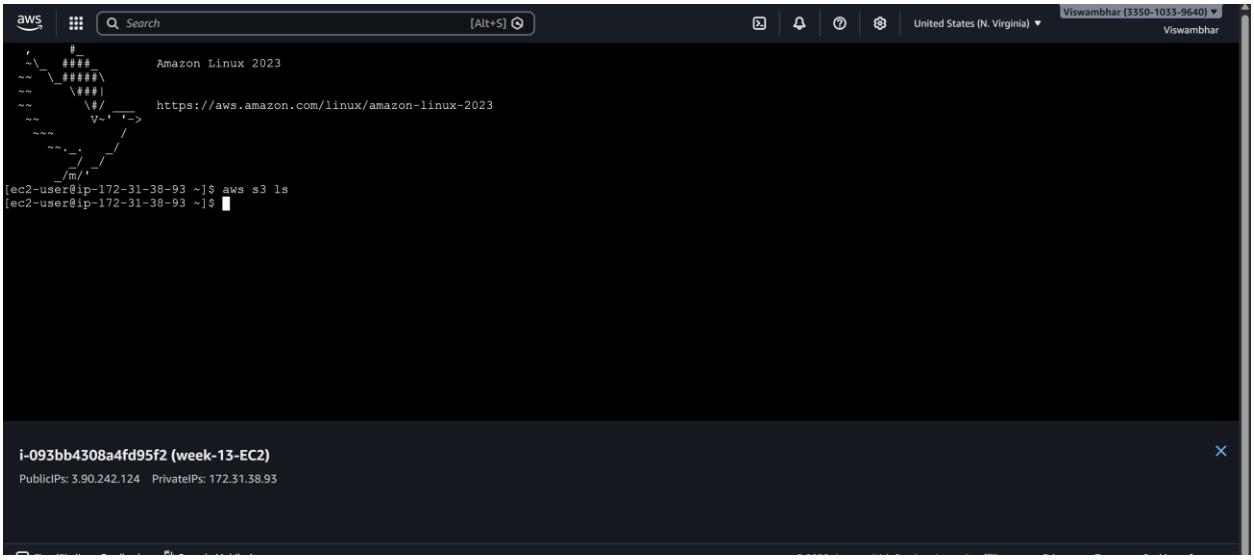
6. Select EC2-S3-Role → Update

7. 

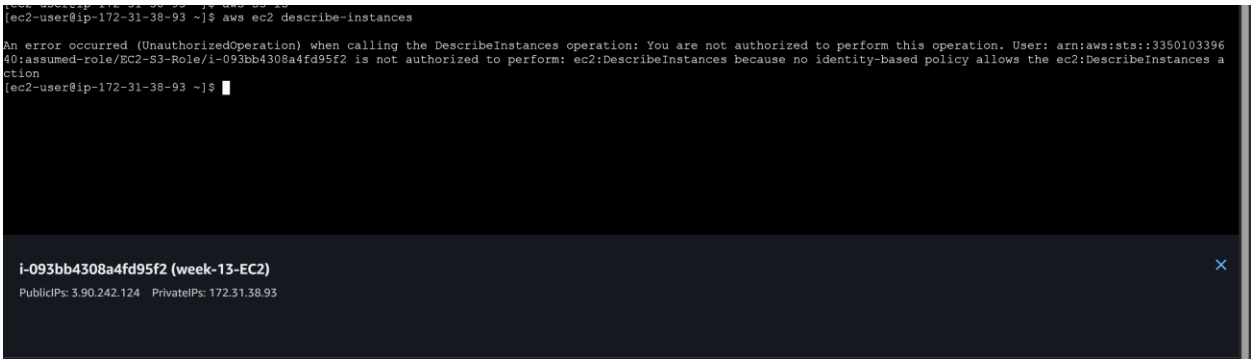
Step 5 — Verify from Instance (No Access Keys!)

SSH into instance and run:

aws s3 ls
aws ec2 describe-instances

- aws s3 ls Works
- 

- aws ec2 describe-instances **Unauthorized (no EC2 permission)**

- 

Reason: Instance receives **temporary credentials** from IAM Role automatically.

Scenario-based questions

Q1. A new developer joins your team and needs access to EC2 and S3. How will you grant permissions securely?
Answer:
Create an IAM user (or preferably use IAM Identity Center), add the developer to a group such as "Developers," and attach a least-privilege policy that allows only required EC2 and S3 actions. Enable MFA and avoid using root credentials.

Q2. A user requires access to AWS resources for only two days. How can you provide temporary access?
Answer:
Use temporary credentials through an IAM Role and AWS Security Token Service (STS), or grant time-bound access using a role session with expiration. Access automatically expires after the defined period.

Q3. A developer accidentally exposed AWS access keys on GitHub. What immediate steps will you take?
Answer:
Immediately:

1. Deactivate or delete the exposed access keys.
2. Generate new keys if needed.
3. Check AWS CloudTrail logs for misuse.
4. Review affected resources.
5. Rotate credentials and follow incident response procedures.

Q4. A user wants to access AWS services using the AWS CLI. How will you enable programmatic access?
Answer:
Create an IAM user (or role), enable programmatic access by generating Access Key ID and Secret Access Key, assign appropriate permissions, and configure it using AWS CLI with aws configure.

Q5. Fifty employees require the same S3 permissions. How will you manage this efficiently?
Answer:
Create an IAM group, attach the required S3 policy to the group, and add all 50 users to that group. This is easier than assigning permissions individually.

Q6. You must remove access permissions for 20 users at once. What is the easiest method?
Answer:
Remove those users from the IAM group or detach the policy from the group. Group-based permission management makes bulk changes easy.

Q7. HR, Developers, and Admin teams need different levels of access. How will you organize permissions?
Answer:
Create separate IAM groups such as HR, Developers, and Admins. Attach different policies to each group based on job roles using Role-Based Access Control (RBAC).

Q8. An EC2 instance must access S3 without storing access keys. What is the correct approach?
Answer:
Attach an IAM Role (Instance Profile) to the Amazon EC2 instance with S3 permissions. The instance gets temporary credentials automatically.

Q9. Account A needs to access resources in Account B. How can cross-account access be configured?
Answer:
Create an IAM Role in Account B, allow Account A as a trusted entity, and let users in Account A assume that role using STS.

Q10. A Lambda function must read and write data to DynamoDB. How will you assign permissions?
Answer:

Attach an execution role to the AWS Lambda function with a policy allowing required DynamoDB actions like GetItem, PutItem, and UpdateItem.

Q11. A user needs administrator privileges for a short time. How can you grant this safely?
Answer:
Use Just-In-Time access by allowing the user to temporarily assume an admin role with limited session duration through STS, instead of permanently attaching AdministratorAccess.

Q12. A user should access only one specific S3 bucket. How do you restrict access?
Answer:
Create an IAM policy granting access only to that bucket's ARN and attach it to the user. Do not allow wildcard (*) access to all buckets.

Q13. Users should be prevented from deleting any AWS resources. How can you enforce this?
Answer:
Use explicit Deny policies for delete actions (like Delete*) or apply AWS Organizations Service Control Policies (SCPs) to block deletions.

Q14. A user should be allowed to log in only during office hours. How can this be implemented?
Answer:
Use an IAM policy with time-based conditions using aws:CurrentTime to allow access only during office hours.

Q15. Access must be allowed only from the office network IP address. How can this be controlled?
Answer:
Use an IAM policy condition with aws:SourceIp and allow access only from the organization's public office IP range.

Q16. What are the best practices to manage IAM permissions securely?
Answer:
Best practices include:

1. Follow least privilege principle.
2. Use IAM roles instead of long-term access keys.

3. Enable MFA.
4. Use groups for permission management.
5. Rotate credentials regularly.
6. Monitor activity using AWS CloudTrail.
7. Avoid using the root account.
8. Review and audit permissions regularly.
9. Use temporary credentials whenever possible.
10. Apply SCPs and explicit denies for critical protection.